Ściągawka VIP: Transformery i duże modele językowe

Afshine Amidi i Shervine Amidi

13 września 2026

Ta ściągawka przedstawia treść książki Super Study Guide: Transformers & Large Language Models, która na 250 stronach, z około 600 ilustracjami, szczegółowo omawia poniższe zagadnienia. Więcej: superstudy.guide.

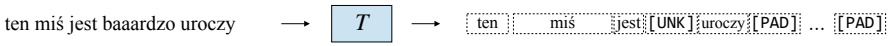
1 Podstawy

1.1 Tokeny

□ **Definicja** — Token to niepodzielna jednostka tekstu, np. słowo, podsłowo lub znak, należąca do z góry ustalonego słownika.

Uwaga: Token [UNK] reprezentuje nieznane fragmenty tekstu, a token dopełniający [PAD] wypełnia puste pozycje, aby sekwencje wejściowe miały jednakową długość.

□ **Tokenizator** — Tokenizator T dzieli tekst na tokeny o wybranym poziomie szczegółowości.



Główne typy tokenizatorów:

Typ	Zalety	Wady	Ilustracja
Słowo	• Prosta interpretacja • Krótka sekwencja	• Duży słownik • Brak obsługi odmian słów	rudy kot
Podsłowo	• Wykorzystuje rdzenie • Intuicyjne osadzenia	• Dłuższa sekwencja • Bardziej złożona tokenizacja	ru ##dy kot
Znak Bajt	• Bez problemu nieznanych tokenów • Mały słownik	• Znacznie dłuższe sekwencje • Trudna interpretacja wzorców na tak niskim poziomie	r u d y k o t

Uwaga: Byte-Pair Encoding (BPE) i Unigram to popularne tokenizatory na poziomie podsłów.

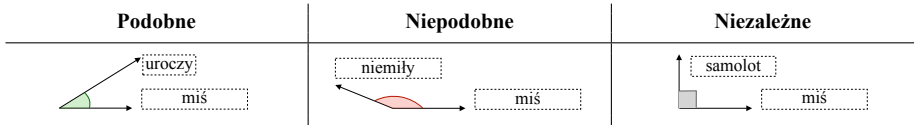
1.2 Osadzenia (embeddingi)

□ **Definicja** — Osadzenie (embedding) to liczbowa reprezentacja elementu, np. tokenu lub zdania, opisana wektorem x o n rzeczywistych składowych.

□ **Podobieństwo** — Podobieństwo kosinusowe dwóch tokenów t_1 i t_2 wyraża wzór:

$$\text{similarity}(t_1, t_2) = \frac{t_1 \cdot t_2}{||t_1||\ ||t_2||} = \cos(\theta) \in [-1, 1]$$

Kąt θ określa podobieństwo między dwoma tokenami:

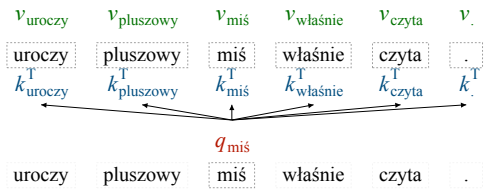


Uwaga: Approximate Nearest Neighbors (ANN) i Locality Sensitive Hashing (LSH) pozwalają efektywnie przybliżać wyszukiwanie podobnych elementów w dużych bazach.

2 Transformery

2.1 Mechanizm uwagi

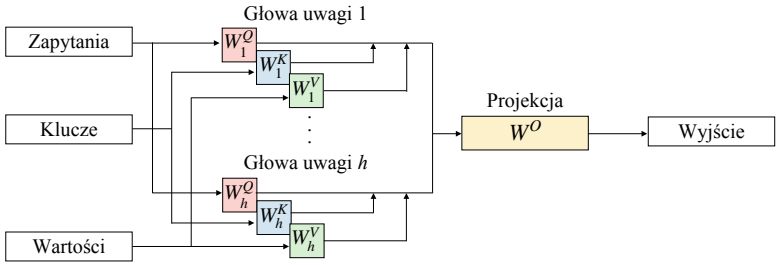
□ **Wzór** — Dla zapytania q chcemy ustalić, na który klucz k należy skierować uwagę, uwzględniając powiązaną z nim wartość v.



Uwagę można efektywnie obliczyć za pomocą macierzy Q, K, V, zawierających odpowiednio zapytania q, klucze k i wartości v. Wymiar kluczy oznaczamy przez d_k:

$$\text{attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

□ **MHA** — Warstwa uwagi wielogłowicowej (Multi-Head Attention, MHA) oblicza uwagę w wielu głowach, a następnie rzutuje wynik na przestrzeń wyjściową.

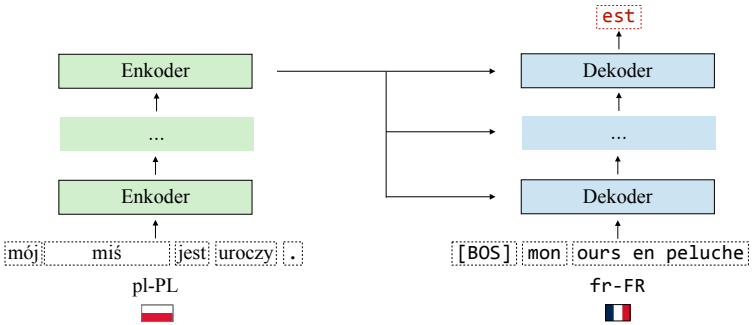


Składa się z h głów uwagi i macierzy W^Q, W^K, W^V, które rzutują wejście na zapytania Q, klucze K i wartości V. Projekcję wyjściową realizuje macierz W^O.

Uwaga: Grouped-Query Attention (GQA) i Multi-Query Attention (MQA) to odmiany MHA zmniejszające koszt obliczeń przez współdzielenie kluczy i wartości między głowami.

2.2 Architektura

□ **Przegląd** — Transformer to przełomowy model oparty na samouwadze (self-attention), złożony z enkoderów i dekoderów. Enkodery tworzą osadzenia reprezentujące znaczenie wejścia; dekodery wykorzystują je do przewidywania kolejnego tokenu.

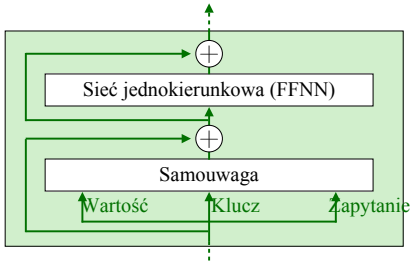


Uwaga: Transformer powstał jako model do tłumaczenia, ale dziś jest szeroko wykorzystywany także w wielu innych zastosowaniach.

□ **Elementy** — Enkoder i dekoder to dwa podstawowe elementy Transformera, pełniące różne role:

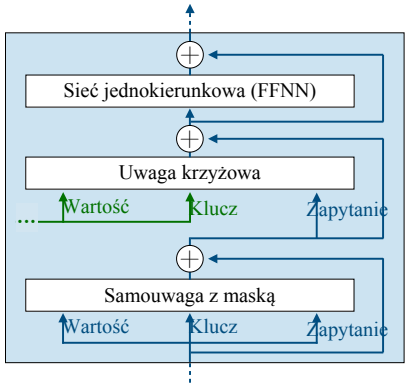
Enkoder

Osadzenia enkodera reprezentują znaczenie wejścia.



Dekoder

Osadzenia dekodera reprezentują znaczenie wejścia i dotychczas przewidzianego wyjścia.

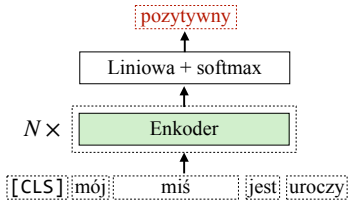


□ **Osadzenia pozycyjne** — Informują o pozycji tokenu w sekwencji. Mogą być zdefiniowane z góry albo wyuczone na podstawie danych.

Uwaga: Rotary Position Embeddings (RoPE) to popularna, efektywna odmiana osadzeń pozycyjnych: obraca wektory zapytań i kluczy, kodując ich względne pozycje.

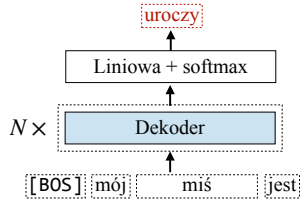
2.3 Warianty

□ **Tylko enkoder** — BERT (Bidirectional Encoder Representations from Transformers) jest modelem złożonym ze stosu enkoderów. Przyjmuje tekst i tworzy osadzenia oddające jego znaczenie, wykorzystywane później np. w zadaniach klasyfikacji.



Na początku sekwencji dodaje się token [CLS], którego osadzenie ujmuje znaczenie zdania. Jest często używane w dalszych zadaniach, np. analizie sentymentu.

□ **Tylko dekoder** — GPT (Generative Pre-trained Transformer) to autoregresyjny Transformer złożony ze stosu dekoderów. W odróżnieniu od BERT-a i jego pochodnych traktuje wszystkie problemy jako zadania przekształcania tekstu w tekst.



Większość współczesnych czołowych LLM-ów ma architekturę z samym dekoderem, np. modele z rodzin GPT, DeepSeek, Mistral i Gemma.

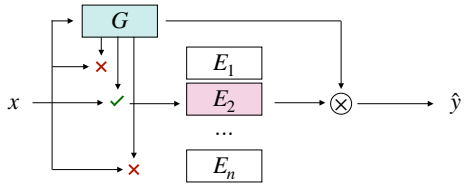
Uwaga: Modele enkoder–dekoder, np. T5, również są autoregresyjne i mają wiele cech wspólnych z modelami z samym dekoderem.

3 Duże modele językowe

3.1 Przegląd

□ **Definicja** — Duży model językowy (Large Language Model, LLM) to model oparty na Transformerze o silnych zdolnościach przetwarzania języka naturalnego (NLP). Zwykle ma miliardy parametrów i jest trenowany na dużych zbiorach danych.

□ **Mieszanina ekspertów** — Mixture of Experts (MoE) aktywuje podczas inferencji tylko część neuronów. Wykorzystuje bramkę G oraz ekspertów $E_1, \ldots, E_n$.



$$\hat{y} = \sum_{i=1}^{n} G(x)_i E_i(x)$$

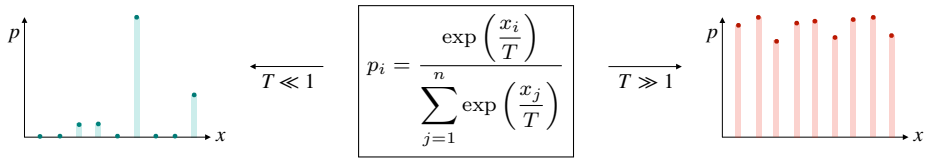
LLM-y oparte na MoE stosują tę bramkę w sieciach jednokierunkowych (feed-forward, FFNN), kierując każdy token do wybranej grupy ekspertów.

3.2 Promptowanie

□ **Długość kontekstu** — To maksymalna liczba tokenów mieszczących się na wejściu modelu. Zwykle wynosi od dziesiątek tysięcy do milionów tokenów.

Uwaga: W miarę zapełniania okna kontekstu jakość odpowiedzi LLM-a często maleje. Zjawisko to nazywa się degradacją kontekstu (context rot).

□ **Próbkowanie przy dekodowaniu** — Tokeny losuje się z przewidzianego rozkładu prawdopodobieństwa p_i, kontrolowanego przez hiperparametr temperatury T.



Uwaga: Wyższa temperatura sprzyja bardziej kreatywnym odpowiedziom, a niższa — bardziej deterministycznym.

4 Trenowanie LLM-ów

4.1 Trening wstępny

□ **Strata entropii krzyżowej** — Cross-Entropy (CE) to funkcja straty, która karze model za niskie prawdopodobieństwo p_i przypisane poprawnemu kolejnemu tokenowi i:

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{V} y_i \log p_i$$

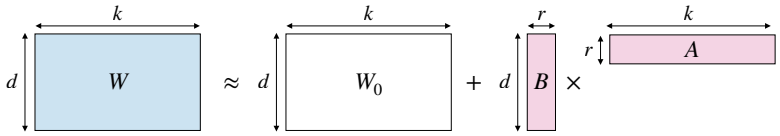
□ **Prawa skalowania** — To empiryczne zależności opisujące przewidywalną poprawę jakości według prawa potęgowego wraz ze wzrostem modelu, ilości danych i nakładu obliczeń.

4.2 Dostrajanie (finetuning)

□ **SFT** — Nadzorowane dostrajanie (Supervised FineTuning, SFT) dopasowuje zachowanie wstępnie wytrenowanego modelu do zadania, korzystając z wysokiej jakości par wejście–wyjście odpowiadających temu zadaniu.

Uwaga: Jeśli dane SFT mają formę instrukcji, etap ten nazywa się dostrajaniem do instrukcji.

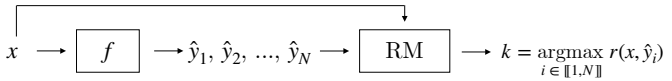
□ **PEFT** — Parameter-Efficient FineTuning (PEFT) obejmuje metody SFT efektywne parametrowo. Low-Rank Adaptation (LoRA) przybliża uczone wagi W: zamraża W_0, a zamiast nich uczy macierze o niskim rzędzie A i B:



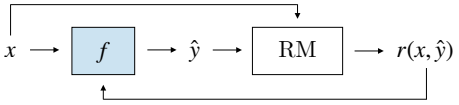
Uwaga: Inne techniki PEFT to m.in. dostrajanie prefiksów i wstawianie warstw adapterów.

4.3 Dostrajanie do preferencji

□ **Model nagrody** — Reward Model (RM) ocenia, jak wynik $\hat{y}$ dla wejścia x odpowiada pożądanemu zachowaniu. Próbkowanie Best-of-N (BoN), zwane też próbkowaniem z odrzucaniem, wybiera za pomocą RM najlepszą z N odpowiedzi.



□ **Uczenie ze wzmocnieniem** — Reinforcement Learning (RL) wykorzystuje RM do aktualizacji modelu f na podstawie nagród za wygenerowane wyniki. Jeśli RM odzwierciedla preferencje ludzi, jest to Reinforcement Learning from Human Feedback (RLHF).

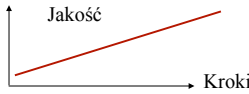
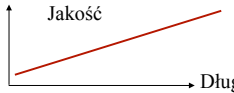


Uwaga: Metody nadzorowane, np. Direct Preference Optimization (DPO), łączą rolę modelu nagrody i RL w jednym nadzorowanym kroku.

4.4 Rozumowanie

□ **Łańcuch rozumowania** — Chain-of-Thought (CoT) to proces, w którym model dzieli złożony problem na serię kroków pośrednich. Pomaga mu to wygenerować poprawną odpowiedź końcową.

□ **Model rozumujący** — Korzysta ze śladów rozumowania CoT do rozwiązywania trudniejszych zadań matematycznych, programistycznych i logicznych. Zdolność rozumowania rozwija się przez dwa rodzaje skalowania:

	Opis	Ilustracja
Skalowanie treningu	Dłuższy trening RL uczy model tworzenia śladów rozumowania CoT przed udzieleniem odpowiedzi.	
Skalowanie inferencji	Dłuższe rozumowanie przed odpowiedzią, wymuszane np. słowami „Poczekaj” (Wait) zwiększającymi budżet rozumowania.	

□ **OPD** — On-Policy Distillation (OPD) to metoda dalszego treningu wykorzystująca trajektorie próbkowane z uczonego modelu. Silniejszy model odgrywający rolę nauczyciela zapewnia dla nich nadzór na poziomie poszczególnych tokenów.

Uwaga: W przeciwieństwie do rzadkich nagród dla całych sekwencji OPD dostarcza gęsty sygnał uczenia od nauczyciela dla tokenów w całej wygenerowanej trajektorii.

5 Uczenie ze wzmocnieniem w LLM-ach

5.1 Podstawy

□ **Ujęcie** — W zastosowaniach RL do LLM-ów:

- **Agentem** jest model językowy. Jego polityka (strategia działania) określa rozkład prawdopodobieństwa działań, czyli kolejnych tokenów.
- **Stan** to prompt i dotychczas wygenerowane tokeny.
- **Działanie** to kolejny wylosowany token.
- **Nagroda** ocenia wygenerowaną odpowiedź według pożądanego zachowania lub wyniku zadania.

□ **Gradienty polityki** — Ta rodzina algorytmów RL aktualizuje politykę na korzyść działań o wyższej skumulowanej nagrodzie R (return) w próbkowanych trajektoriach. Ponieważ wartości R mogą się znacznie różnić, poziom odniesienia $b(s)$ pozwala ocenić wynik względem oczekiwań: $R - b(s)$. Zmniejsza to wariancję aktualizacji.

Uwaga: REINFORCE jest przykładem algorytmu gradientu polityki.

5.2 PPO

□ **Funkcja wartości** — Szacuje oczekiwaną przyszłą nagrodę dla danego stanu lub pary stan–działanie przy bieżącej polityce.

□ **Algorytm** — Proximal Policy Optimization (PPO) zwiększa nagrody i utrzymuje model blisko bazowego, by ograniczyć manipulowanie nagrodą (reward hacking).

Dla ilorazu prawdopodobieństw $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ otrzymujemy:

$$\boxed{\mathcal{L}_{\text{PPO}} = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t,\ \text{clip}\left(r_t(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_t\right)\right]}$$

5.3 GRPO

□ **Weryfikowalne nagrody** — Są wyliczane automatycznie na podstawie obiektywnie sprawdzalnych wyników, np. dokładnych odpowiedzi, wykonania kodu lub formalnych dowodów.

□ **Algorytm** — Group Relative Policy Optimization (GRPO) wyznacza przewagi $\hat{A}_i$, porównując nagrody R_i w grupie G wylosowanych odpowiedzi. Nie wymaga osobnego modelu wartości.

Dla znormalizowanej przewagi $\hat{A}_i = \frac{R_i - \text{mean}(R_1,\ldots,R_G)}{\text{std}(R_1,\ldots,R_G)}$ otrzymujemy:

$$\boxed{\mathcal{L}_{\text{GRPO}} = \frac{1}{G}\sum_{i=1}^{G}\mathbb{E}_t\left[\min\left(r_{i,t}(\theta)\hat{A}_i,\ \text{clip}\left(r_{i,t}(\theta), 1-\epsilon, 1+\epsilon\right)\hat{A}_i\right) - \beta D_{\text{KL}}\left(\pi_\theta \| \pi_{\text{ref}}\right)\right]}$$

Uwaga: Istnieją rozszerzenia łagodzące ograniczenia GRPO: Dr. GRPO koryguje obciążenie związane z długością, a DAPO poprawia stabilność i eksplorację dzięki zmodyfikowanemu przycinaniu (clipping) i dynamicznemu próbkowaniu.

6 Systemy LLM

6.1 Kernele obliczeniowe

□ **FlashAttention** — To dokładna metoda obliczania uwagi, zoptymalizowana pod kątem GPU. Operacje macierzowe wykorzystują szybką statyczną pamięć RAM (SRAM), a wyniki są zapisywane do wolniejszej pamięci o wysokiej przepustowości (HBM).

Uwaga: W praktyce zmniejsza to zużycie pamięci i przyspiesza obliczenia.

□ **PagedAttention** — Zarządza pamięcią podręczną KV w blokach o stałym rozmiarze. Ogranicza fragmentację i umożliwia efektywne współdzielenie pamięci między sekwencjami.

6.2 Trening rozproszony

□ **Równoległość** — Rozdziela trening LLM-a między wiele urządzeń przez podział danych, obliczeń lub elementów modelu.

□ **Rodzaje** — Główne formy równoległości:

- **Danych:** kopie modelu na urządzeniach; podział partii danych treningowych.
- **Tensorowa:** podział obliczeń wewnątrz pojedynczych warstw między urządzenia.
- **Potokowa:** podział warstw modelu na etapy umieszczone na różnych urządzeniach.
- **Ekspertów:** rozmieszczenie ekspertów MoE na urządzeniach i kierowanie tokenów.

Uwaga: Trening LLM-ów na dużą skalę zwykle łączy kilka form równoległości.

6.3 Optymalizacja inferencji

□ **Pamięć podręczna KV** — Przechowuje obliczone wcześniej wektory kluczy i wartości podczas dekodowania. Eliminuje powtarzanie tych samych obliczeń uwagi przy generowaniu kolejnych tokenów.

Uwaga: Buforowanie promptów pozwala ponownie używać wpisów KV dla powtarzających się prefiksów.

□ **Prefill i decode** — Inferencja LLM-a ma dwie fazy: prefill przetwarza równolegle wszystkie tokeny promptu i inicjalizuje pamięć KV; decode generuje nowe tokeny autoregresyjnie, jeden po drugim.

□ **Dekodowanie spekulatywne** — Przyspiesza generowanie: mniejszy model pomocniczy proponuje kilka tokenów, które model docelowy następnie weryfikuje równolegle.

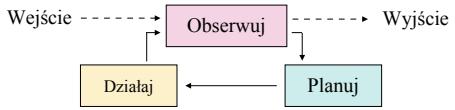
Uwaga: Proponowanie tokenów małym modelem jest tanie. Model docelowy może sprawdzić kilka propozycji w jednym przejściu w przód, zamiast generować je pojedynczo, co przyspiesza dekodowanie.

7 Agenci AI

7.1 Przegląd

□ **Definicja** — Agent AI to system, który autonomicznie realizuje cele i zadania w imieniu użytkownika. Może w tym celu wykorzystywać różne łańcuchy wywołań LLM-a.

□ **ReAct** — Reason + Act (ReAct), czyli rozumowanie i działanie, to schemat wykorzystujący łańcuchy wywołań LLM-a do realizacji złożonych zadań:



Schemat obejmuje następujące kroki:

- **Obserwuj:** podsumuj dotychczasowe działania i jasno określ, co już wiadomo.
- **Planuj:** określ zadania do wykonania i narzędzia, które należy wywołać.
- **Działaj:** wykonaj operację przez API lub wyszukaj informacje w bazie wiedzy.

Uwaga: Ocena systemów agentowych jest trudna. Można jednak badać zarówno wejścia i wyjścia poszczególnych elementów, jak i całe łańcuchy wywołań systemu.

□ **Warstwa wykonawcza (harness)** — To otoczenie modelu zarządzające kontekstem, narzędziami, stanem i pętlą działania agenta.

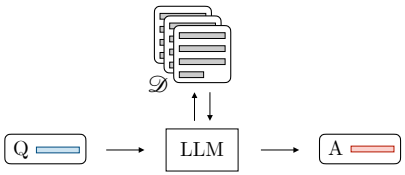
Uwaga: Przykłady to Codex, Claude Code i Pi.

7.2 Wyszukiwanie informacji

□ **Retriever** — Wybiera istotne dokumenty z bazy wiedzy przez wyszukiwanie oparte na słowach kluczowych (sparse retrieval), osadzeniach (dense retrieval) lub połączeniu obu metod.

□ **Reranker** — Porządkuje wyszukane dokumenty według dokładniejszego modelu trafności, poprawiając jakość kontekstu przekazywanego LLM-owi.

□ **RAG** — Retrieval-Augmented Generation (RAG) wzbogaca generowanie o wyszukiwanie zewnętrznej wiedzy potrzebnej do odpowiedzi. Przydaje się zwłaszcza do uwzględniania informacji nowszych niż dane z treningu wstępnego LLM-a.



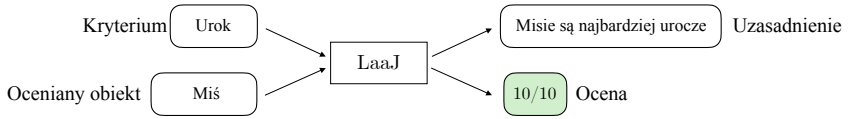
Dla bazy wiedzy D i pytania retriever pobiera najbardziej istotne dokumenty, następnie wzbogaca nimi prompt, na podstawie którego LLM generuje odpowiedź.

Uwaga: Jakość RAG silnie zależy od podziału dokumentów na fragmenty, jakości wyszukiwania i liczby dokumentów dołączonych do kontekstu.

8 Ewaluacja

8.1 LLM jako sędzia

□ **LLM jako sędzia** — LLM-as-a-Judge (LaaJ) wykorzystuje model językowy do oceniania wyników według zadanych kryteriów. Może też generować uzasadnienie oceny, co ułatwia jej interpretację.



W odróżnieniu od starszych metryk, np. ROUGE (Recall-Oriented Understudy for Gisting Evaluation), LaaJ nie wymaga tekstu referencyjnego i nadaje się do różnych zadań. Oceny silnego modelu (np. GPT-6) dobrze korelują z ocenami ludzi; metoda wymaga bowiem zdolności rozumowania.

Uwaga: LaaJ ułatwia szybkie rundy ewaluacji. Trzeba jednak monitorować zgodność jego ocen z ocenami ludzi, aby wykrywać ewentualne rozbieżności.

□ **Typowe błędy oceny** — LaaJ może wykazywać następujące błędy systematyczne (bias):

	Efekt pozycji	**Efekt rozwlekłości**	**Autopreferencja**
Problem	Preferuje pierwszą pozycję w porównaniach parami	Preferuje bardziej rozwlekłe odpowiedzi	Preferuje odpowiedzi wygenerowane przez siebie
Rozwiązanie	Uśrednij wyniki przy losowej kolejności	Dodaj karę za długość odpowiedzi	Użyj sędziego opartego na innym modelu bazowym

Uwaga: Powyższa lista błędów systematycznych nie jest wyczerpująca.

8.2 Ewaluacja agentów

□ **Komponent** — Ocena na poziomie komponentów bada jakość poszczególnych modułów agenta, np. wyszukiwania, wyboru narzędzi lub wykonywania działań.

Uwaga: Metryki: dokładność, precyzja, czułość, skuteczność wywołań narzędzi i opóźnienie.

□ **Trajektoria** — Ocena na poziomie trajektorii bada całą sekwencję obserwacji, decyzji i działań podjętych przez agenta.

Uwaga: Metryki: skuteczność realizacji zadań, liczba kroków lub wywołań narzędzi, częstość błędów, opóźnienie i koszt.

9 Dyfuzyjne modele językowe

9.1 Paradygmat

□ **ARM** — Model autoregresyjny (AutoRegressive Model, ARM) generuje sekwencję token po tokenie, warunkując każdy nowy token wszystkimi poprzednimi.

□ **Dyfuzyjny LLM** — Model dLLM tworzy tekst przez iteracyjne odszumianie całej sekwencji. W odróżnieniu od niego ARM generuje tokeny kolejno, od lewej do prawej. Wyróżnia się dwa główne rodzaje dLLM-ów:

- **Dyfuzja ciągła:** odszumia ciągłe osadzenia tokenów, a następnie odwzorowuje wynik na tokeny dyskretne.
- **Dyfuzja dyskretna:** odszumia tokeny bezpośrednio w przestrzeni dyskretnej.

Uwaga: W dyfuzyjnych LLM-ach dominuje obecnie dyfuzja dyskretna, zwłaszcza maskowana: tokeny są stopniowo maskowane i odtwarzane.

9.2 Trenowanie

□ **Cel** — Model uczy się odtwarzać oryginalne tokeny z zakłóconej sekwencji przy losowo wybranym poziomie szumu.

□ **Proces dyfuzji w przód** — Stopniowo zakłóca czysty tekst, np. zastępując coraz większy odsetek tokenów symbolem [MASK].

9.3 Inferencja

□ **Strategia** — Podczas inferencji dyfuzyjny LLM zaczyna od zakłóconej lub zamaskowanej sekwencji i wielokrotnie przewiduje tokeny, aż powstanie pełna odpowiedź.

Uwaga: Dyfuzyjne LLM-y naturalnie wspierają generowanie w dowolnej kolejności i uzupełnianie luk, ponieważ nie muszą działać wyłącznie od lewej do prawej.

□ **Harmonogram dekodowania** — Określa, które przewidziane tokeny zostają przyjęte lub ponownie zamaskowane w każdym kroku odszumiania, równoważąc szybkość i jakość.

Doprecyzowania redakcyjne

Poniższe uwagi nie są częścią tekstu autorów. Wyjaśniają uproszczenia oryginału i wybory lokalizacyjne.

1.1 — Znaki i bajty. W tabeli użyto przykładu ASCII: w UTF-8 każdy jego znak zajmuje jeden bajt. Puste pole oznacza spację. Litera „ł” zajmuje dwa bajty: C5 82. Brak nieznanych tokenów w tokenizacji znakowej zależy od słownika; pełny słownik bajtów pozwala zakodować dowolny tekst UTF-8.

1.2 — Podobieństwo. „Niezależne” w rysunku oznacza tu wektory prostopadłe, o zerowym podobieństwie kosinusowym — nie dowód niezależności statystycznej.

4.3 — DPO. DPO optymalizuje model bez osobnego, jawnego modelu nagrody i bez pętli treningu RL. Określenie „łączy RM i RL” w oryginale jest skrótem myślowym.

4.3 — Best-of-N. Best-of-N wybiera odpowiedź o najwyższej ocenie spośród N propozycji. Nazwa „próbkowanie z odrzucaniem” nie oznacza tu klasycznego algorytmu accept–reject.

5.2 — PPO. Iloraz we wzorze porównuje nową politykę ze starą, która wygenerowała próbki, niekoniecznie z modelem bazowym. Samo przycinanie nie gwarantuje wyeliminowania reward hacking. Pokazany dodatni cel PPO należy maksymalizować.

5.3 — GRPO. Wzór na przewagę wymaga obsługi przypadku zerowego odchylenia standardowego nagród w grupie. Schemat nie pokazuje wszystkich szczegółów implementacji.

8.1 — LLM jako sędzia. Zgodność z ocenami ludzi zależy od zadania, kryteriów i modelu sędziego. Brak potrzeby tekstu referencyjnego nie oznacza, że ocena jest zawsze wiarygodna.

Źródła: RFC 3629 (UTF-8); artykuł DPO; OpenAI Spinning Up — PPO; artykuł MT-Bench.

Informacje o polskiej wersji

Roboczy przekład przygotowany dla Jakuba Florkowskiego, 3 października 2026. Nie został zatwierdzony przez autorów oryginału. Przed publikacją wymaga weryfikacji językowej i merytorycznej. Autorzy oryginału: **Afshine Amidi i Shervine Amidi**. Zachowano strukturę, wzory i większość grafiki oryginalnej sześciostronicowej ściągawki CME 295. Rysunki uwagi i tokenizacji dostosowano do polskich przykładów. Przekład poddano przeglądowi porównawczemu i językowemu; nie zastępuje to zatwierdzenia przez autorów.

Źródło: repozytorium CME 295 — Transformers & Large Language Models
Wersja źródła: c086dee78457511da4a084f25339502bfc4ae899
Kurs: cme295.stanford.edu
Książka: superstudy.guide/transformers-large-language-models

Licencja oryginału — MIT License

MIT License
Copyright (c) 2025 Afshine Amidi and Shervine Amidi
Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:
The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.
THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.